

6.2 Debugging

Debugging is the process of identifying, locating, and correcting defects (bugs) found during testing. It is NOT the same as testing — testing finds bugs, debugging fixes them.

The Debugging Process

1. Reproduce the bug consistently.
2. Identify the root cause in the source code.
3. Fix the defect.
4. Verify the fix (re-test).
5. Perform regression testing to ensure no new bugs were introduced.

Debugging Strategies

Brute Force	Adding print statements or memory dumps throughout the code to find the defect. Common but time-consuming.
Backtracking	Starting from the point where the error is observed and tracing back through the code to find the root cause.
Cause Elimination	Hypothesize possible causes, design tests to confirm or eliminate each hypothesis.
Program Slicing	Using automated tools to identify the subset of program statements that could affect a variable's value at a specific point.

Common Types of Bugs

- Syntax errors – Violations of language grammar rules (caught by compiler).
- Logic errors – Incorrect algorithm or wrong condition (hardest to find).
- Runtime errors – Errors that occur during program execution (e.g., division by zero).
- Off-by-one errors – Loop runs one too many or too few times.
- Memory leaks – Program fails to release memory after use.
- Concurrency bugs – Issues in multi-threaded programs (race conditions, deadlocks).

7. Software Implementation Techniques: Coding Practices & Refactoring

7.1 Good Coding Practices

Good coding practices improve code quality, readability, maintainability, and reduce defects. They form the foundation of professional software development.

7.1.1 Code Readability and Clarity

- Use meaningful, descriptive names for variables, functions, and classes.
 - Write one statement per line; avoid overly complex expressions.
 - Limit function length (typically < 50 lines per function).
 - Add comments to explain 'why', not 'what' — code should be self-explanatory.
- Follow consistent indentation and formatting standards.

7.1.2 Defensive Programming

Defensive programming anticipates potential errors and handles them

- gracefully:
- Validate all inputs at boundaries (entry points of modules).
 - Handle all error conditions explicitly; never silently ignore errors.
 - Use assertions to check preconditions and postconditions.
 - Design for failure — assume something will go wrong.

7.1.3 Coding Standards

Coding standards enforce consistency across a team:

- Naming conventions (e.g., camelCase for variables, PascalCase for classes).
- Standard file header with author, date, purpose, and revision history.
- Maximum line length (usually 80–120 characters).
 - Documentation of all public APIs and interfaces.

7.2 Refactoring

Refactoring is the process of improving the internal structure of existing code without changing its external behavior. The goal is to make the code cleaner, more readable, and easier to maintain.

Key Principle: Refactoring improves the design of existing code. It does NOT add new functionality or fix bugs — it restructures code to make it better.

Why Refactor?

- Code becomes harder to understand over time ('code rot').
- To remove duplicated code.
- To improve performance.
- To make future changes easier (reduce technical debt).
- To apply better design patterns.

Common Refactoring Techniques

Extract Method	Move a code fragment into a separate method. Reduces long methods.
-----------------------	--

Rename Variable/Method	Change names to be more descriptive and meaningful.
Replace Magic Numbers	Replace numeric literals (e.g., 3.14159) with named constants (PI).
Remove Duplicate Code	Consolidate duplicated logic into a single method.
Decompose Conditional	Break complex IF-ELSE into smaller, well-named methods.
Encapsulate Field	Make a public field private and provide getter/setter methods.
Replace Conditional with Polymorphism	Use inheritance/polymorphism instead of complex switch/if-else chains.
Introduce Explaining Variable	Place a complex expression into a well-named variable.
Move Method/Field	Move a method or field to the class where it logically belongs.
Consolidate Conditional	Combine multiple conditionals with the same result into one.

Code Smells (Indicators that Refactoring is Needed)

- Long Method – Method does too much.
- Large Class – Class has too many responsibilities.
- Duplicate Code – Same logic in multiple places.
- Long Parameter List – Method takes too many parameters.
- Dead Code – Code that is never executed.
- Comments explaining bad code – If you need a comment to explain code, refactor the code.

Refactoring vs. Rewriting

Refactoring	Rewriting
Incremental improvement of existing code	Discarding old code and writing from scratch
Preserves external behavior	May change behavior
Lower risk	Higher risk
Continuous process	One-time activity

8. Maintenance and Reengineering – BPR Model

8.1 Software Maintenance

Software maintenance is the process of modifying a software system after delivery to correct faults, improve performance, or adapt it to a changed environment. It is the longest phase of the SDLC.

IEEE Definition: Software maintenance is the modification of a software product after delivery to correct faults, to improve performance or other attributes, or to adapt the product to a modified environment.

Types of Software Maintenance

Corrective Maintenance	Fixing bugs and defects found after deployment. Corrects errors in logic, code, or design. (~20% of maintenance effort)
Adaptive Maintenance	Modifying the software to work in a new or changed environment (new OS, hardware, regulations). (~25% of maintenance effort)
Perfective Maintenance	Enhancing functionality, improving performance, or adding new features based on user requests. (~50% of maintenance effort)
Preventive Maintenance	Making changes to prevent future failures. Includes code restructuring and documentation updates. (~5% of maintenance effort)

Challenges in Software Maintenance

- Understanding legacy code that is poorly documented.
- High cost – maintenance can consume 60-80% of total software cost.
- Staff turnover – original developers may not be available.
- Ripple effects – a change in one module can break others.

8.2 Software Reengineering

Software reengineering is the examination and alteration of a software system to reconstitute it in a new form. It is used to improve an aging system without completely rebuilding it from scratch.

When to Reengineer?

- Software is difficult to maintain due to poor structure.
- System performance is unacceptable.
- Software must be adapted to a new technology platform.
- High defect rates despite repeated bug fixing.

8.3 Business Process Reengineering (BPR) Model

Business Process Reengineering (BPR) is the fundamental rethinking and radical redesign of business

processes to achieve dramatic improvements in critical performance measures such as cost, quality, service, and speed.

Key Characteristics of BPR

- Fundamental: Questions why things are done the way they are.
- Radical: Involves completely redesigning processes from the ground up.
- Dramatic: Seeks quantum leaps in performance, not incremental improvements.
- Process-focused: Centers on business processes, not tasks, jobs, or people.

BPR Model – Steps

Step	Phase	Description
1	Business Modeling	Model the information flow among business functions. Define what information drives each process.
2	Data Modeling	Identify data objects, attributes, and their relationships within the business context.
3	Process Modeling	Define how data objects are transformed within the business processes.
4	Application Prototyping	Use CASE tools to rapidly build prototypes of new applications.
5	Code Generation	Generate application code using CASE tools where possible.
6	Testing and Turnover	Test reengineered software and transition it to the business.

BPR vs. Traditional Improvement

Business Process Reengineering	Continuous Improvement (TQM)
Radical, breakthrough change	Incremental, step-by-step change
Top-down, management-driven	Bottom-up, employee-driven
Focuses on core processes	Focuses on all processes
High risk, high reward	Low risk, steady gains

9. Reengineering Process Model – Reverse & Forward Engineering

9.1 Software Reengineering Process Model

The software reengineering process model describes the activities involved in transforming an existing legacy system into a better-structured, more maintainable system. It involves a systematic sequence of activities.

Activities in the Reengineering Process Model

#	Activity	Description
1	Inventory Analysis	Catalog all existing software components. Identify candidates for reengineering based on age, criticality, and maintenance cost.
2	Document Restructuring	Improve or create documentation for poorly documented systems.

Page 5 | Prepared for B.Tech CSE Students

SRM University AP | CSE 306 - Software Engineering and Project Management | Unit 4: Testing & Maintenance

#	Activity	Description
3	Reverse Engineering	Analyze existing code to extract design and specification information.
4	Code Restructuring	Improve the code structure without changing external behavior (similar to refactoring).
5	Data Restructuring	Understand the existing data model and modify it to support better design.
6	Forward Engineering	Take the design recovered from reverse engineering and create a new, improved system.

9.2 Reverse Engineering

Reverse Engineering is the process of analyzing software to create a representation of the software at a higher level of abstraction. It reconstructs the design and specification from existing code.

Reverse Engineering does NOT change the software. It only creates a representation of what the software does, to aid understanding.

Levels of Abstraction in Reverse Engineering

Level	What is Recovered
Implementation Level	Code restructuring, removal of dead code, improving readability.
Design Level	Data flow diagrams, structure charts, class diagrams from source code.
Specification Level	Algorithms, data structures, functional behavior from design.
Requirements Level	Business rules, functional requirements from the specification.

Activities in Reverse Engineering

- Redocumentation – Creating or improving documentation from existing source code.
- Design Recovery – Recovering the system design through code analysis, often using CASE tools.
- Program Understanding – Making existing code understandable to human engineers.
- Data Reverse Engineering – Recovering data models from existing databases.

9.3 Forward Engineering

Forward Engineering (also called Renovation or Reclamation) takes the information recovered through reverse engineering and uses it to reconstruct the existing system, incorporating new requirements and improving quality.

Forward Engineering vs. Normal Development

Page 6 | Prepared for B.Tech CSE Students

SRM University AP | CSE 306 - Software Engineering and Project Management | Unit 4: Testing & Maintenance

Forward Engineering	New Development
Starts from an existing legacy system	Starts from scratch
Informed by recovered design/specifications	Informed by fresh requirements only
Uses existing business logic as foundation	Business logic defined from requirements
Lower risk – existing functionality preserved	Higher risk – complete rewrite

9.4 The Relationship: Reverse + Forward Engineering

The complete reengineering cycle combines reverse engineering and forward engineering:

Reverse Engineering	Restructuring	Forward Engineering
Existing code → Design	Improving code structure / data	Design → New improved code
Understand the old system	Clean up and reorganize	Build the new system
Input: Legacy code	Input: Recovered artifacts	Output: Reengineered system

The complete path: Existing Code → Reverse Engineering → Design Recovery → Forward Engineering → Improved New System.

Economics of Reengineering-I

- A cost/benefit analysis model for reengineering has been proposed by Sneed [Sne95]. Nine parameters are defined:
 1. P_1 = current annual maintenance cost for an application.
 2. P_2 = current annual operation cost for an application.
 3. P_3 = current annual business value of an application.
 4. P_4 = predicted annual maintenance cost after reengineering.
 5. P_5 = predicted annual operations cost after reengineering.
 6. P_6 = predicted annual business value after reengineering.
 7. P_7 = estimated reengineering costs.
 8. P_8 = estimated reengineering calendar time.
 9. P_9 = reengineering risk factor ($P_9 = 1.0$ is nominal).
 10. L = expected life of the system.

Economics of Reengineering-II

- The cost associated with continuing maintenance of a candidate application (i.e., reengineering is not performed) can be defined as

$$C_{\text{maint}} = [P_3 - (P_1 + P_2)] \times L$$

- The costs associated with reengineering are defined using the following relationship:

$$C_{\text{reeng}} = [P_6 - (P_4 + P_5) \times (L - P_8) - (P_7 \times P_9)]$$

- Using the costs presented in equations above, the overall benefit of reengineering can be computed as

$$\text{cost benefit} = C_{\text{reeng}} - C_{\text{maint}}$$

9.5 CASE Tools in Reengineering

Computer-Aided Software Engineering (CASE) tools support reengineering activities:

- Code analyzers – Parse source code to identify structure and dependencies.
- Re-documentation tools – Automatically generate documentation from code.
- Code restructuring tools – Transform unstructured code into structured form.
- Data restructuring tools – Assist in database schema recovery and migration.
- Forward engineering tools – Generate code from recovered design models.

10. Important Questions

Short Answer Questions

6. Define software testing. What are its objectives?
7. What is cyclomatic complexity? How is it calculated? Give one formula.
8. Differentiate between white box testing and black box testing.
9. What is equivalence class partitioning? Give an example.
10. What is regression testing? When is it performed?

11. Differentiate between verification and validation.
12. What are test stubs and test drivers? Where are they used?
13. Define software maintenance. What are its types?
14. What is refactoring? List any four refactoring techniques.
15. What is reverse engineering? What levels of abstraction does it produce?

Long Answer Questions

16. Explain basis path testing with a suitable example. How do you calculate cyclomatic complexity?
17. Describe the various types of control structure testing: condition testing and loop testing with examples.
18. Explain boundary value analysis and equivalence class partitioning with examples.
19. Describe the different levels of testing: unit testing, integration testing, validation testing, and system testing.
20. Explain the various types of integration testing strategies with their advantages and disadvantages.
21. What is software reengineering? Explain the reengineering process model in detail.
22. Distinguish between reverse engineering and forward engineering. Explain with the help of a diagram.
23. Explain the Business Process Reengineering (BPR) model. What are its key characteristics?
24. What is software maintenance? Explain the different types of maintenance with examples.
25. Write short notes on: (a) Debugging strategies (b) System testing types (c) Alpha and Beta testing. (12 Marks)

11. Quick Revision Summary

Topic / Term	Key Point to Remember
Testing Objective	Find defects — not to prove software is correct.
$V(G)$ = Cyclomatic Complexity	$E - N + 2P$; or Decision Points + 1; Low = simple code, >10 = refactor.
White Box Testing	Internal view; tests code paths; done by developers.
Black Box Testing	External view; tests functions from spec; done by testers.
Equivalence Partitioning	Divide input into valid/invalid classes; test one from each.
Boundary Value Analysis	Test at and just beyond boundaries (a-1, a, a+1, b-1, b, b+1).
Regression Testing	Re-run tests after code changes to ensure no new bugs.
Test Stub vs Driver	Stub = dummy called module; Driver = dummy calling module.
Integration Strategies	Big Bang, Top-Down, Bottom-Up, Sandwich.
Validation Testing	Are we building the right product? SRS-based testing.

Topic / Term	Key Point to Remember
System Testing Types	Performance, Stress, Security, Recovery, Usability, Compatibility.
Debugging vs Testing	Testing finds bugs; Debugging locates and fixes them.
Refactoring	Improve code structure without changing behavior. Reduces technical debt.
Code Smells	Long method, duplicate code, large class, dead code = refactor needed.
Maintenance Types	Corrective (fix bugs), Adaptive (new env.), Perfective (enhance), Preventive.
BPR	Fundamental, radical redesign of business processes for dramatic improvement.
Reverse Engineering	Code → Design → Specification (understanding old system).
Forward Engineering	Design → Code (building new improved system from recovered artifacts).
Reengineering	Reverse Engineering + Restructuring + Forward Engineering.